Code-Layout, Readability and Reusability

Date	29 September 2026
Team ID	_____
Project Name	LegalEase: AI-Powered Legal Document Generator

Maximum Marks: 5 Marks

Code-Layout, Readability and Reusability: This document evaluates the LegalEase codebase in terms of structure, readability and reusability. The project uses Streamlit for the frontend, FastAPI for the backend, Gemini integration for document generation, and separate formatting utilities for DOCX, PDF and HTML output.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Use consistent Python indentation across all modules.
2	Proper File Structure	Files and folders are logically organized	Yes	main.py, routes.py, ai_core and formatting modules are separated logically.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	document_type, parties, terms, dates and DocumentRequest are descriptive.
4	Function / Method Names	Functions are descriptively named	Yes	generate_document, sanitize_text, format_docx, format_pdf and format_html_preview are clear.
5	Code Comments	Inline and block comments explain logic	Partial	Architecture is documented; complex implementation sections can use more inline comments.
6	Modular Design	Code is split into reusable functions/modules	Yes	AI generation, API routes and document formatting are separated into reusable components.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Formatting and generation responsibilities are handled by dedicated utilities.
8	Error Handling	Exceptions and errors are handled gracefully	Partial	Pydantic validation and modular API design are used; production-level exception handling can be strengthened.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	GeminiDocumentGenerator	Python / Gemini API	Used by document-generation logic for different legal document types.	High

2	DocumentRequest	Python / Pydantic	Validates inputs for the / generate API endpoint.	High
3	format_docx(text, doc_type)	Python / python-docx	Creates formatted DOCX output with logo, headings and terms table.	High
4	format_pdf(text, doc_type)	Python / FPDF	Creates branded PDF output with headers, footers and bullet terms.	High
5	format_html_preview(text)	Python / HTML / Streamlit	Creates the styled editable preview.	High
6	sanitize_text(text)	Python	Cleans generated text before formatting/export.	High
7	FastAPI /generate	Python / FastAPI	Connects frontend input to Gemini generation.	Medium
8	Streamlit UI	Python / Streamlit	Collects document type, parties, terms and dates and provides downloads.	Medium

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Clear separation of frontend, backend, AI integration and formatting utilities.
Readability	4	Descriptive names and a straightforward input-to-generation flow improve readability.
Reusability	5	Core generation and formatting functions can support multiple legal document types.
Documentation / Comments	4	Project documentation is detailed; additional inline comments can improve maintainability.
Overall Score	**4.5 / 5**	**Strong modular organization and reusable components, with room to strengthen comments and explicit error handling.**

Source basis: LegalEase project document and the supplied Code-Layout, Readability and Reusability template.